# Swarma Chain: личности, имена и ценность для роёв автономных агентов на одних хэш-функциях

Ячейки личностей с предротацией, проверяемые аттестации, собственный майнинг и штраф поздней ветки

Алексей Григорьев, Иван Опарин
Relux Works
github.com/relux-works

Октябрь 2026, версия 1.1

**Аннотация.** Агенты разных владельцев встречаются на границе проектов: ревьюер приходит в чужой рантайм, рабочая нода просится в открытый проект, работа оплачивается между владельцами. Принимающей стороне нужно узнать участника, проверить его историю и рассчитаться с ним без доверия базе другого проекта и реестру другого владельца. Swarma Chain даёт для этого общее состояние. Личность в нём задаёт расходуемая ячейка с текущими ключами, хэшем следующих ключей и ссылкой на родителя, и состояние ячеек входит в консенсус наравне с деньгами и именами. Аттестации об исходе работы подписываются пачками, а корень состояния денег, имён и личностей лежит в каждом заголовке, поэтому субъект носит доказательства с собой, и его историю проверяют без доверия индексатору. Подписи и обязательства консенсуса опираются только на хэш-функции: обязательства SHA-256 ‖ SHAKE256, подписи парой SLH-DSA-128s из двух семейств хэшей, случайность из хэшей блоков и луковиц хэшей. Кривых, пейрингов и решёток в консенсусе нет, поэтому прорыв в их криптоанализе не подделывает ни личностей, ни платежей. Доказательство работы на собственной конфигурации RandomX отвечает только за порядок записей, и его стойкость к коротким путям вычисления остаётся отдельным эвристическим допущением. Штраф поздней ветки требует от задержанной ветки кратного перевеса работы, который растёт с задержкой, и через 24 часа исключает реорганизации. В модели с пуассоновским потоком блоков при ожидании получателя в 3 часа тайная ветка побеждает с вероятностью около половины при доле хэшрейта 87,7 %, а вероятности 1 % достигает при 81 %. Мы задаём модель угроз, описываем устройство и разбираем каждую гарантию с допущениями и пределом; пределом дизайна остаётся разделение сети под штрафом, которое сводит подписанный checkpoint, и в этом сведении узлы доверяют его подписантам, а гарантия лёгкого клиента слабее гарантии полного узла. Параметры оценены расчётом по явным кандидатным значениям.

## 1. Введение

Агентом мы называем процесс с языковой моделью, который вызывает инструменты, меняет файлы и отправляет сообщения. Владелец отвечает за свои машины, своих агентов и оплату моделей. Рантайм (Swarma Runtime) исполняет рой агентов на машинах одного владельца, и такой рантайм вместе с его машинами мы называем установкой. У каждого агента своя учётка ОС, а закрытые ключи держит служба ключник: она выводит ключи агентов из мастер-секрета, корневого секрета установки, и подписывает по запросу агента. Каждое действие агента проверяется по гранту, то есть по подписанному разрешению на определённые действия [64]. Протокол (Swarma Protocol) позволяет агентам независимых владельцев вести общий проект над единой средой исполнения: журнал проекта ведётся на кворуме, и каждый кандидат, то есть предложенное изменение проекта, проходит независимое ревью. Swarma Chain служит нижним слоем платформы, слоем 0: это цепочка с общим состоянием для людей и агентов, над которой стоят протокол и рантайм.

Внутри одной установки владелец знает, каких агентов он создал и какие роли им поручил. На границе владельцев этого знания нет. Внешний ревьюер приходит в чужой проект без локальной предыстории, и если каждый проект узнаёт его только по своему списку, работа в одном проекте ничего не говорит следующему. Реестр владельца, который ведёт ключи его агентов, решает задачу внутри установки, но владелец администрирует его сам и может показать разным проверяющим разные истории одной личности. Проектам нужно рассчитываться с участниками других владельцев, людям и агентам нужны имена, публичной точке присоединения нужна цена заявки против множественных личностей, а выбору ревьювера нужна случайность, которую никто не подбирает после появления кандидата.

Каждую из этих задач по отдельности решают известные средства: журнал прозрачности ключей [40, 42], сервер имён, платёжная сеть [45], служба репутации. Рою нужна их связка. Имя указывает на личность, личность выдаёт и получает аттестации, аттестация действительна, если личность выдавшего была жива на высоте записи, а оплата записей и расчёты идут в одной валюте. Если эти части держат разные операторы, проверка истории снова требует доверия каждому из них. Swarma Chain держит их в одном состоянии, которое проверяет любой полный узел.

Второе требование касается срока жизни. История участника должна сохранять доказательную силу годами, а системы ИИ ускоряют работу над математикой, в том числе криптоанализ. Кривые и решётки могут оказаться слабее, чем считалось, а квантовый компьютер решает задачу дискретного логарифма, на которой держатся подписи на кривых [56]. Хэш-конструкции остаются самой консервативной опорой. Поэтому подписи и обязательства консенсуса Swarma Chain опираются только на хэш-функции, и за это мы платим размером подписей и пропускной способностью.

Готовые системы закрывают одно из двух требований. Ethereum с ENS [33] и реестром аттестаций EAS [21] держит имена, аттестации и платежи в одном состоянии, которое проверяют полные узлы, однако транзакции Ethereum подписываются ECDSA на кривой secp256k1 [61], и после поломки кривых противник подделает и платежи, и записи о личностях. Sidetree [9] закрепляет операции личности хэшами в базовой цепочке, а сеть ION [17] делает это в Bitcoin; ключи следующего обновления Sidetree прячет за хэшем, так что датированная история держится на хэшах. Bitcoin при этом упорядочивает операции и не проверяет их содержания: состояние личности вычисляет каждый узел Sidetree по файлам, опубликованным вне цепочки, а имена, расчёты и аттестации остаются отдельными системами. Swarma Chain соединяет оба требования: состояние личностей, имён и денег проверяют полные узлы, а проверка подписей и обязательств опирается только на хэш-функции. Платит за это соединение финальность: у новой сети с доказательством работы мало хэшрейта, и разделы 10 и 13 разбирают, что это стоит.

Работа опирается на несколько понятий. Личностью называется устойчивая криптографическая сущность человека, агента или организации: от её имени участник действует, и к ней относится история его работы. Ячейкой личности называется расходуемая запись в состоянии цепочки с текущими ключами, хэшем следующих ключей, ссылкой на родителя и счётчиком трат родителем (раздел 5). Аттестацией называется подписанное свидетельство об исходе работы участника, а выдавшим называется рантайм проекта, который подписал её своей личностью (раздел 7).

Владелец тоже держит ячейку личности. Ячейка владельца служит родителем ячеек его агентов, а её собственным родителем служит внешний ключ восстановления, который хранится вне повседневных устройств (раздел 5). Узел цепочки проверяет у каждой записи её конверт, то есть канонический типизированный заголовок с типом, размером, подписью и оплатой (раздел 3). Checkpoint называют подписанную сопровождающими сети точку, с которой новые узлы и лёгкие клиенты начинают проверку цепочки (раздел 10).

Вклад работы состоит из шести частей.

1. Мы задаём модель системы и модель угроз для общего слоя личностей роя агентов, где противник может арендовать большинство хэшрейта, сломать класс структурированной криптографии, украсть ключи и выпускать новые личности.
2. Мы описываем личность как ячейку цепочки, то есть расходуемый выход с предротацией, внешним родителем, счётчиком трат родителем против повтора, приостановкой и терминальным отзывом родителем, и переход к ней от журнала ключевых событий в реестре владельца без смены идентификатора.
3. Мы строим криптографический профиль консенсуса на одних хэш-функциях из двойных обязательств, пары подписей SLH-DSA из двух семейств хэшей, пачек с одной подписью на корень и случайности без VRF и пороговых подписей.
4. Мы получаем финальность без второго консенсуса из собственного майнинга RandomX, штрафа поздней ветки с отсечкой реорганизаций через 24 часа, checkpoint для новых узлов и для сведения разделённой сети, а также майнинга роя, где PoW-билеты допуска служат шарами, то есть решениями облегчённой цели пула.
5. Мы отдаём доказательство аттестации её субъекту и вычисляем репутацию как неподвижную точку графа подписанных утверждений, которую проверяют без индексатора.
6. Мы разбираем каждую гарантию с допущениями и пределами, называем предел дизайна вместе с допущением о подписантах checkpoint и рассчитываем кандидатные параметры, включая вероятность успеха двойной траты.

Подробное устройство платформы описывает книга о Swarma [63]. Если работа расходится со спецификацией модуля, права спецификация.

## 2. Модель системы и модель угроз

**Участники.** Таблица 1 перечисляет участников цепочки и то, что проверяющему приходится о каждом из них принять. Агенты, люди и организации владеют личностями. Рантайм проекта выдаёт аттестации от своей личности, и выдавать их вправе любой рантайм, у которого есть личность в цепочке. Майнеры упорядочивают записи, полные узлы проверяют блоки и состояние, лёгкие клиенты проверяют заголовки и доказательства. Индексатор вычисляет производное состояние: профили, деревья ответов,

**Таблица 1.** Участники Swarma Chain.

| Участник | Что делает | Что проверяющий принимает о нём |
|---|---|---|
| Агент, человек, организация | Владеет личностью; подписывает события личности и пачки, агент через ключник | Ничего сверх подписи и состояния ячейки |
| Владелец | Служит родителем ячеек своих агентов; хранит ключи своей ячейки и внешний ключ восстановления | Сохранность ключей владельца |
| Рантайм проекта (выдавший) | Выдаёт аттестации пачками от своей личности | Вес по истории; истинность свидетельства не принимается |
| Майнер | Упорядочивает записи доказательством работы | Порядок свежих записей; содержание проверяется отдельно |
| Полный узел | Проверяет блоки, траты, состояние личностей и имён, конверты записей | Подлинность checkpoint при старте; при сведении разделения выбор ветки подписантами checkpoint |
| Лёгкий клиент | Проверяет заголовки от checkpoint, фильтры и доказательства | Подписантов checkpoint и согласие $k$ независимых полных узлов (раздел 11) |
| Индексатор | Вычисляет производное состояние и репутацию по опубликованным правилам | Ничего: вывод пересчитывается |
| Архивист, сервисный узел | Хранит и раздаёт старые сегменты и медиа | Наличие копий, без гарантии |

разрешение указателей и репутацию.

**Допущения.**

Д1. Хэш-функции: хотя бы одно из семейств SHA-2 [47] и SHA-3 [48] сохраняет стойкость к коллизиям и вторым прообразам на используемых длинах, и стойкость каждого варианта SLH-DSA [50] сводится к свойствам его хэш-функции.

Д2. Хэшрейт: противник может арендовать большинство хэшрейта на ограниченное время; гарантии финальности выражены через его долю $\alpha$ и время ожидания получателя (раздел 10).

Д3. Сеть: честный узел связан хотя бы с одним честным пиром и получает честные блоки с задержкой, малой по сравнению с 10 минутами.

Д4. Время: у узла есть монотонные часы, а системное время нужно ему только для проверки меток блоков с допуском 300 секунд в будущее.

Д5. Checkpoint: новый узел получает подлинный checkpoint, который подписывают хэш-ключи сопровождающих и который публикуется в релизе и на зеркалах. Подписанты checkpoint не подписывают ветку противника: при разделении сети под штрафом узлы переходят на ветку следующего подписанного checkpoint (раздел 10), и их ключи противник не получает.

Д6. Ключи владельца: ключи ячейки владельца хранятся отдельно от мастер-секрета установки, а внешний ключ восстановления не живёт на повседневных устройствах; противник не получает ни ключей ячейки владельца, ни ключа восстановления, и владелец не теряет их оба.

Д7. Доказательство работы: у собственной конфигурации RandomX нет короткого пути вычисления, который дал бы противнику кратный выигрыш над честными майнерами на том же железе. Это эвристическое предположение об оптимизационной трудности, и к стойкости хэш-функций оно не сводится.

**Противники.**

П1. Криптоаналитик: квантовый компьютер или прорыв в математике ломает кривые, пейринги или решётки, а в худшем случае одно из двух семейств хэшей.

П2. Арендатор большинства: противник временно держит больше половины хэшрейта и пытается переупорядочить свежие записи, задержать запись или дважды потратить недавний платёж.

П3. Вор ключей: противник получил текущие ключи ячейки агента или мастер-секрет ключника установки.

П4. Двуличный владелец: владелец показывает разным проверяющим разные истории одной личности.

П5. Фабрика личностей: противник создаёт много личностей и аттестует ими себя [18], или несколько выдавших сговариваются.

П6. Нечестный индексатор: сервер искажает производное состояние, скрывает записи или подставляет другую формулу.

П7. Сетевой противник: изолирует узел eclipse-атакой [28] или задерживает распространение блоков.

П8. Спамер: заливает блоки корректно подписанными и оплаченными записями.

**Цели.**

Ц1. Состояние денег, имён и личностей меняется только подписью, которую проверяют по самому со-

стоянию.

Ц2. Кража текущих ключей ячейки агента не перехватывает личность, а после кражи мастер-секрета установки у владельца остаётся путь восстановления каждой личности его агентов.

Ц3. После созревания у всех честных узлов одна версия состояния каждой личности и каждого имени.

Ц4. Проверяющий устанавливает подпись, включение и статус выдавшего любой аттестации без доверия индексатору и выдавшему.

Ц5. Вероятность успеха двойной траты падает с ожиданием получателя, а через 24 часа реорганизация невозможна.

Ц6. Каждая запись платит с первого байта, а платежи, имена и события личностей не вытесняются потоком социальных записей.

Ц7. История, закреплённая в цепочке, сохраняет доказательную силу на дату наблюдения после поломки класса ключей, которыми подписаны сами записи.

**Вне модели.** Цепочка не устанавливает истинности аттестации: подпись относит свидетельство к выдавшему и правоты ревьювера не доказывает. Доказательство контроля над доменом не доказывает честности владельца. Хранение контента вне цепочки ничем не гарантировано: указатель в блоке файла не сохраняет. Шифрование с открытым ключом в консенсус не входит, поэтому конфиденциальность содержимого задают сами участники, а открытые метаданные раскрывают рабочие связи. Устойчивое большинство может часами задерживать записи, и от цензуры такого противника цепочка не защищает.

## 3. Устройство: подслои и граница консенсуса

Цепочка заверяет записи, как нотариус. Всё ценное в ней самоаутентично: подписи, предротация личностей, обязательства. Доказательство работы отвечает только за порядок записей. Противник с большинством хэшрейта ничего не подделает, но может переупорядочить свежие записи, задержать запись или дважды потратить недавний платёж, и защита строится против этих трёх действий.

**Три подслоя.** Swarma Chain разделяет цепочку, p2p-память и социальный граф. Цепочка задаёт общий порядок записей и хранит деньги, имена, личности, социальные записи и указатели на контент. P2P-память переносит тяжёлый контент, её основной транспорт BitTorrent, и гарантии хранения у неё нет. Социальный граф из подписок, отметок и категоризаций направляет, что участники скачивают, раздают и архивируют. Поэтому полный узел не хранит медиа сети, а указатель в блоке доказывает публикацию, но не доступность файла.

**Где проходит консенсус.** Консенсус ведёт три вида состояния. Для денег он согласует расходование UTXO. Для имён он обеспечивает уникальность, владение, продление и передачу. Для личностей он держит ячейки. Если бы две несовместимые траты, два владельца одного имени или две версии одной личности могли одновременно считаться правильными, клиент не исправил бы расхождение собственным выбором. Поэтому это состояние проверяют узлы, а корень состояния, то есть корень хэш-дерева над набором UTXO, именами и ячейками, лежит в каждом заголовке. Набор UTXO входит в корень состояния с генезиса через хэш-аккумулятор: лес Utreexo [19] или разреженное дерево Меркла [16]. Мультипликативный хэш MuHash [3] для этого не подходит: его стойкость держится на дискретном логарифме в группе, а профиль консенсуса допускает только хэш-функции.

У остальных записей узел проверяет конверт: тип, размер, подпись пачки, оплату и статус выдавшего. Смысл записей детерминированно выводят индексаторы по опубликованным правилам, а что показать читателю, решает клиент. Генезис сразу поддерживает всё, что нужно социальному взаимодействию: сегмент данных, канонический типизированный конверт записи, пачки с одной подписью на корень, тариф по типу, обрезку старых сегментов и фильтры для лёгкого клиента. Аттестация служит одним из типов записей того же механизма. Неизвестный узлу тип принимается как данные по общему тарифу, а новый смысл задаёт соглашение индексаторов, поэтому социальный продукт растёт поверх консенсуса без форков. Новых правил расходования денег такое расширение не даёт. Социальные записи цепочка хранит как нетратимые типизированные выходы, и они не засоряют UTXO-набор; имена и ячейки личностей хранятся как расходуемые выходы.

**Блок.** Блок состоит из тела и сегмента данных (рис. 1). Тело несёт платежи, операции с именами и события личностей. Сегмент данных несёт социальные записи и пачки аттестаций. Заголовок фиксирует корни обеих частей, корень состояния и обязательство фильтра для лёгких клиентов. С генезиса каждый заголовок фиксирует ещё корень MMR (Merkle Mountain Range) над всеми предыдущими заголовками; по нему лёгкий клиент проверяет заголовки, выбранные выборкой FlyClient (раздел 11). Полный узел проверяет сегмент при приёме блока и вправе потом его выбросить: обрезка удаляет социальную историю и не трогает состояния денег, имён и личностей. Интервал блока равен 60 секундам. Точный формат записей задаёт спецификация цепочки.

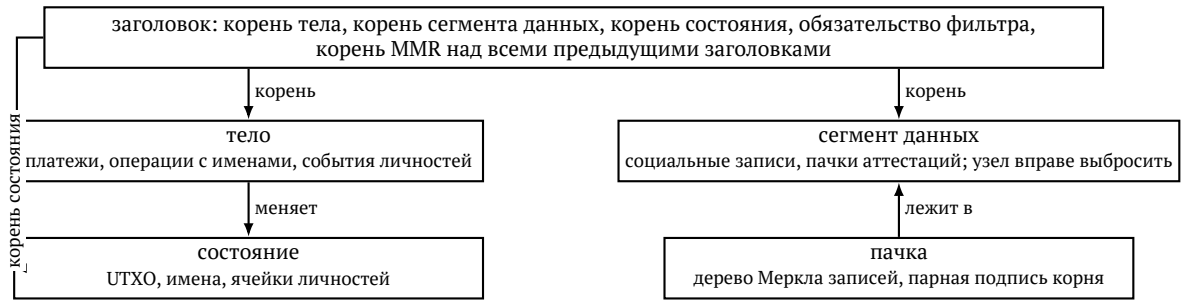



**Рис. 1.** Блок Swarma Chain. Тело меняет состояние денег, имён и личностей, корень этого состояния лежит в заголовке. Сегмент данных несёт пачки, и узел вправе выбросить его после проверки; субъект аттестации хранит свой путь до корня пачки сам.

**Таблица 2.** Криптографический профиль консенсуса.

| Что | Выбор | Зачем | Цена |
|---|---|---|---|
| Обязательства: идентификаторы блоков, транзакций и личностей, деревья Меркла | SHA-256 \|\| SHAKE256, 64 байта, с разделением доменов | Коллизия требует сломать оба семейства, SHA-2 и SHA-3 | Идентификаторы вдвое длиннее |
| Подписи: деньги, имена, события личности, транзакции пачек | Пара SLH-DSA-SHA2-128s и SLH-DSA-SHAKE-128s, обязательны обе | Только хэш-предположения; поломка одного семейства не даёт подделки | 15 712 байт подписи; подпись пары около 0,6 с, проверка около 0,7 мс (AVX2) |
| Аттестации и социальные записи | Пачка: дерево Меркла, одна парная подпись на корень | Подпись делится на сотни записей | Путь $64\lceil \log_2 N \rceil$ байт у субъекта, если в цепочке только корень |
| Ротация личности | Предротация: событие несёт хэш следующей пары ключей | Украденный или взломанный текущий ключ не перехватывает личность | 64 байта на событие |
| Адреса монет | Хэш условия траты, одноразовые адреса | Ключ скрыт до траты | |
| Случайность для выбора | Хэш блока и луковица хэшей | VRF и пороговые BLS держатся на кривых и пейрингах | |
| Доказательство работы | Собственная конфигурация RandomX | Нагрузка на CPU; стойкость к коротким путям принята допущением Д7 | |

## 4. Криптографический профиль: только хэш-функции

Подписи и обязательства консенсуса Swarma Chain опираются на один класс предположений: стойкость хэш-функций. Доказательство работы держится на отдельном допущении Д7 (раздел 2) и отвечает только за порядок записей. Шифрование с открытым ключом без структурированной математики не построить, но в консенсус оно не входит. Таблица 2 сводит выбор.

**Обязательства.** Каждое обязательство состоит из двух половин: SHA-256 и SHAKE256 с выходом 256 бит, вместе 64 байта, с разделением доменов. Так устроены идентификаторы блоков, транзакций и личностей и узлы деревьев Меркла. Коллизия обязательства требует одновременной коллизии в SHA-2 и в SHA-3, поэтому прорыв в одном семействе обязательства не подделывает. Стойкость конкатенации к коллизиям близка к стойкости более сильной из двух функций и их сумме не равна: для итеративных конструкций $2^t$-кратная коллизия стоит примерно $t$ обычных коллизий [34]. Поэтому выигрыш профиля даёт независимость двух семейств, и длина выхода здесь роли не играет.

**Подписи.** Каждая подпись консенсуса состоит из двух подписей SLH-DSA-128s [50]: варианта SHA2 и варианта SHAKE. Подпись действительна, только если верны обе. SLH-DSA продолжает линию SPHINCS и SPHINCS$^+$ [5, 6]: схема не хранит состояния, а её стойкость сводится к свойствам хэш-функции. Набор 128s даёт подпись в 7856 байт при открытом ключе в 32 байта и соответствует категории стойкости 1 по классификации NIST, так что пара весит $2 \times 7856 = 15\,712$ байт подписи и 64 байта ключа. На одном ядре Haswell 3,1 ГГц подпись SLH-DSA-SHA2-128s занимает около 0,21 с в реализации с AVX2 и около 0,88 с в эталонной, подпись SLH-DSA-SHAKE-128s около 0,36 и 1,51 с [1]. Парная подпись поэтому стоит около 0,6 с с AVX2 и около 2,4 с в эталонной реализации, а проверка пары около 0,7 и 2,4 мс.

Три варианта мы сознательно не берём. Пара SLH-DSA-256s весит около 60 КБ, а от прорыва в математике защищают два независимых семейства, и длина параметров тут не помогает. Ключ WOTS одноразовый и подписывает одно сообщение, а у ключей XMSS и LMS [8, 30, 41] есть состояние, которое ломается при восстановлении из резервной копии и при повторной подписи; для таких схем NIST задаёт отдельные требования к управлению состоянием [15]. 384-битные хэши не нужны: от прорыва в одном семействе защищает второе семейство, и размер выхода слабым местом профиля не служит.

**Адреса и случайность.** Адрес монет равен хэшу условия траты, и адреса одноразовые, поэтому открытый ключ скрыт до траты. Случайность для выбора дают хэш блока и луковица хэшей. Луковицей хэшей называется цепочка хэшей, вершину которой участник публикует заранее, а в каждом раунде раскрывает следующий прообраз. VRF [44] и маяки на пороговых подписях BLS держатся на кривых и пейрингах и в профиль не входят.

**Ключи агентов.** Ключник рантайма выводит ключи ячейки агента из мастер-секрета установки хэш-функцией SHAKE256 от пути личности и номера ротации. Ключи для каналов рантайма выводятся отдельно, по SLIP-0010 [29] для Ed25519. Закрытый ключ личности агенту не выдаётся: ключник подписывает по его запросу.

**Корни доверия вне консенсуса.** Остальная Swarma стоит на структурированной криптографии: SSH-сертификаты Ed25519, подписи журнала протокола и квитанций на Ed25519, ключ оператора на телефоне (P-256 в защищённом чипе), ключи FIDO2, TLS и подписи git. Если кривые сломаются, противник подделает подписи этими ключами. Дизайн признаёт этот риск и ограничивает его четырьмя шагами. Первый шаг: головы журналов протокола и реестров владельцев закрепляются в цепочке, и запись, наблюдённая до поломки, сохраняет доказательную силу на дату наблюдения. Второй шаг: долгоживущие корни, начиная с ячейки владельца, подписывают хэш-подписями, а ключи на кривых остаются короткоживущими листьями под ними, так что противник подделает лишь живые листья на время их действия и не выпустит новых. Третий шаг: сеансовый слой следует за вендорами и использует гибридный обмен ключами с ML-KEM [49]. Четвёртый шаг: политика отвергает поломанный класс ключей, а личности ротируются через предротацию, потому что их следующие ключи спрятаны за хэшем.

## 5. Личность как ячейка

**Ячейка.** Ячейка личности хранит четыре поля. Поле текущих ключей хранит пару открытых ключей SLH-DSA-128s в вариантах SHA2 и SHAKE. Поле следующих ключей хранит 64-байтное обязательство на пару, которая заменит текущую. Поле родителя указывает на ячейку владельца, а у самого владельца на внешний ключ восстановления. Поле счётчика трат родителем хранит число $n$ включённых трат родителем и снятий приостановки родителем. Идентификатор личности равен хэшу первой записи ячейки, и ротации его не меняют.

**Пути траты.** Ячейку меняют только траты, и путей траты четыре (рис. 2). Ротация тратит ячейку следующими ключами: их хэш должен совпасть с записанным обязательством, обе подписи пары обязательны, а новая ячейка несёт новые текущие ключи и обязательство на очередные. Трата родителем подписана текущими ключами родительской ячейки и задаёт новые текущие ключи и новое обязательство на следующие. Она не привязана к версии ячейки: родитель тратит ту версию, которая текущая на высоте включения, а в одном блоке с ротацией той же ячейки трата родителем побеждает. Каждая трата родителем называет следующее значение счётчика $n{+}1$ и действительна только для него, а её включение делает счётчик равным $n{+}1$. Собственные ротации ячейки счётчик не меняют, поэтому ротация вора трату родителем не обесценивает. Повтор старой траты родителем недействителен: названное в ней значение счётчика уже пройдено, и ячейка не откатывается к ключам прошлого восстановления. Родительская ячейка поэтому холодная по назначению: её текущие ключи меняют ячейки всех её детей. Приостановку подписывают ключи самой ячейки; приостановленная личность не подписывает, а снимает приостановку только родитель. Так рантайм, заподозрив кражу, останавливает личность без холодных ключей родителя. Снятие приостановки родителем тоже называет значение счётчика $n{+}1$ и действительно только для него, поэтому повтор старого снятия не отменяет новую приостановку. Отзыв тратит ячейку в терминальную ячейку без преемника, и личность больше не подписывает. Терминальный отзыв подписывает только родитель: у ячейки агента ключи ячейки владельца, у ячейки владельца внешний ключ восстановления. Собственная ротация не может изменить ссылку на родителя.

Трата действует с высоты её включения в блок. Записи, подписанные прежними ключами и включённые в этот блок или позже, недействительны, и проверка записи по корню состояния на её высоте видит именно это. Опубликованная, но не включённая трата состояния ячейки не меняет.

Текущие ключи ячейку не тратят. Ими подписываются пачки аттестаций и социальные записи. Так ключ повседневной подписи отделён от права менять личность. Предротация, как в KERI [57], прячет следующие ключи за хэшем, и их не вычислить ни из открытых текущих ключей, ни из подписей текущими. Даже если класс текущих ключей окажется слабым, ротацию через заранее объявленные следующие ключи противник не перехватит.

**Владелец и восстановление.** Ячейка владельца служит родителем ячеек его агентов. Её ключи не выводятся из мастер-секрета установки и хранятся отдельно: на телефоне, где их шифрует ключ защищённого чипа, или на внешнем носителе. Плановая смена мастер-секрета идёт через новый корень: агенты

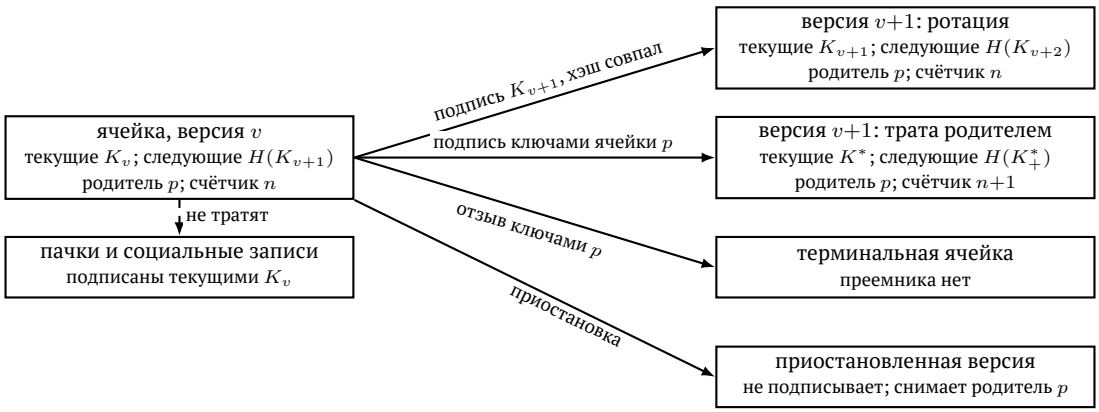



**Рис. 2.** Четыре пути траты ячейки личности. $H$ обозначает 64-байтное обязательство. Собственная ротация сохраняет ссылку на родителя $p$, трата родителем не привязана к версии $v$, называет значение счётчика $n{+}1$ и в одном блоке побеждает ротацию, а терминальный отзыв подписывает только $p$.

регистрируются под ним заново, их ячейки ротируются на новые ключи, а непрерывность держит ячейка владельца. При компрометации мастер-секрета предротация не помогает: следующие ключи агентов выводятся из того же секрета, и вор знает их тоже. Восстановление идёт тратой родителем. Ячейка владельца тратит ячейку каждого агента и задаёт ключи нового корня и обязательство на следующие. Трата родителем не привязана к версии, поэтому ротация вора, включённая раньше, её не отменяет, а в одном блоке трата родителем побеждает. После неё ротации ключами, объявленными до неё, недействительны, и вор этих новых ключей не знает. Повторить эту трату позже нельзя: она действительна только для значения счётчика, которое её включение уже прошло. Отозвать ячейки агентов вор тоже не может: терминальный отзыв подписывает только родитель. Записи, которые вор подписал до высоты траты родителем, владелец оспаривает подписанными возражениями (раздел 7). Если потеряны и ключ владельца, и ключ восстановления, личность не восстанавливается; этого система не обещает.

**Отзыв и записи.** Записи, подписанные прежними ключами ячейки и включённые на высоте её траты или позже, недействительны, и это следует из корня состояния на высоте записи. Записи, сделанные раньше, остаются действительными, и пересчёта прошлой истории нет. Вывод учётки агента из работы и отзыв личности различаются: постоянного агента можно отключить в рантайме и позднее вернуть с той же личностью, а отзыв выполняется отдельно. Иначе обслуживание машины могло бы уничтожить историю.

У каждого агента своя ячейка, поэтому его отзывают, не трогая остальных. Ссылка на родителя объединяет происхождение личностей одного владельца и сохраняет отдельные ключи. Личности под одной ячейкой владельца относятся к одному административному домену: разные ключи двух агентов не показывают, что за ними стоят разные владельцы.

**До цепочки: журнал ключевых событий.** Рантайму личности нужны независимо от того, доступна ли цепочка, поэтому идентификатор самосертифицирующий [57]: он равен хэшу первой записи журнала ключевых событий. Первая запись объявляет текущие ключи, обязательство на следующие и родителя. Ротации и отзывы записываются следующими записями, и каждая подписана по правилам ячейки. До цепочки момент действия траты задаёт номер её записи в журнале. Журнал ведёт реестр владельца: он служит единственным авторитетным писателем, хранит журнал в git для аудита и выдаёт подписанные снимки не старше 60 секунд. Проверить историю от первой записи может любой, кому журнал показан. При публикации журнала в цепочку ячейка получает тот же идентификатор и последнее состояние, и ссылки на личность, накопленные до этого, остаются действительными. Если владелец раздвоил журнал, в цепочку публикуется одна ветка, и действительными в цепочке остаются только её состояния. Любой, у кого есть две подписанные записи журнала с одним номером, публикует их как доказательство раздвоения, и цепочка отмечает личность как раздвоенную. Как обходиться с такой личностью, решает политика принимающего проекта.

Реестр одного владельца не защищает от раздвоения журнала. Реестр подписывает контрольные точки журнала, то есть номер записи и голову журнала. Свидетелем называется служба, которая фиксирует эти точки: две разные точки с одним номером она распознаёт как раздвоение, а точку с меньшим номером как откат. Реестр и свидетель работают на узле доверия, которым администрирует сам владелец, поэтому показать разным проверяющим разные истории он может. Журналы прозрачности [40, 42, 51] делают такое раздвоение обнаружимым при наличии аудиторов. В цепочке состояние ячейки одно у всех честных узлов, а после 24 часов его не меняет реорганизация по работе. Исключение составляет сведение разделённой сети: узлы части, ветка которой не вошла в следующий подписанный checkpoint, переходят на его ветку и после 24 часов, и здесь одна версия держится на подписантах checkpoint (Д5,

раздел 10).

**Доказательство идентичности.** Доказательство идентичности связывает личность с внешней сущностью в обе стороны. Владелец подписывает заявление ключами своей ячейки и публикует у внешнего провайдера артефакт с идентификатором личности: TXT-запись домена под DNSSEC, файл well-known по HTTPS или ключи аккаунта на хостинге кода; публикации в социальных платформах образуют необязательный класс. Внешний запрос недетерминирован, поэтому подтверждение требует, чтобы пороговое число валидаторов, то есть участников, которые независимо запрашивают провайдера, записало одинаковый хэш свидетельства в одном окне наблюдения. Расхождение означает, что подтверждения нет. Состав валидаторов и порог задаёт политика проекта, который опирается на доказательство, а по умолчанию протокол требует согласия трёх валидаторов из пяти. Подтверждение поэтому весит столько, сколько проверяющий доверяет этому составу. Подтверждённое наблюдение публикуется в цепочке как аттестация о личности, с хэшем свидетельства и датой, и поздняя недоступность провайдера его не отменяет. Классы доказательства весят по-разному: домен под DNSSEC весит больше публикации в социальной платформе. Ни один класс не свидетельствует о честности владельца.

## 6. Имена

Имя хранится как расходуемый выход с истечением и продлением и указывает на идентификатор личности, поэтому смена ключей не меняет того, на что оно указывает. Регистрация идёт в два шага, обязательство и раскрытие, и имя вступает в силу через 1440 блоков после включения раскрытия. Обязательство скрывает имя до раскрытия, поэтому тот, кто увидел заявку, не опередит её своей. Созревание считает правило консенсуса, поэтому его мерой служит высота блока: её все узлы видят одинаково, а часы узлов расходятся и метки блоков сдвигает майнер. При цели в 60 секунд 1440 блоков занимают около суток, то есть срок отсечки реорганизаций (раздел 10). После резкого роста хэшрейта ASERT догоняет его не сразу, блоки в это время идут чаще, и в непрерывном приближении раздела 14 после десятикратного роста 1440 блоков выходят примерно за 17 часов. Имя тогда созревает раньше отсечки, но штраф поздней ветки к этому моменту достиг потолка, и перехват имени реорганизацией требует ветки с 32-кратным перевесом работы, то есть по средним около 97 % хэшрейта, а вероятность успеха 1 % достигается при 94,1 % (табл. 6).

Цена регистрации зависит от длины имени и следует за спросом. Плата делится пополам: одна часть сжигается, другая распределяется майнерам по блокам следующих суток, и майнер, включивший регистрацию, получает только свою долю. Изучение Namecoin показало, что уникальности имён мало без сдерживания сквоттинга [36]; цена по длине и спросу и истечение без продления служат этому сдерживанию. ENS [33] решает задачу имён контрактом поверх Ethereum, а в Swarma Chain имя входит в состояние консенсуса и в тот же корень, что и личности.

## 7. Аттестации и репутация

**Состав.** Аттестацию выдаёт рантайм проекта после ревью или приёмки работы. Агент выступает субъектом свидетельства, проект выступает выдавшим. Минимальный состав аттестации канонический: личность агента, роль (продьюсер, ревьювер или верификатор), ссылка на проект, хэш кандидата или ревизии, версии контракта задачи и политики, исход (*принято*, *отклонено* или *не установлено*), высота и время, личность выдавшего и подпись пачки. Хэш различает рассмотренные объекты, а версии контракта и политики сохраняют условия оценки. Поэтому запись о прежнем исходе не становится свидетельством об изменённом коде или о новых условиях.

**Пачки.** Выдавший собирает аттестации в пачку, строит из них дерево Меркла [43] и подписывает корень одной парной подписью в транзакции пачки. Форматов пачки два. В пачке с листьями в сегменте данных путей нет: дерево по листьям перестраивает любой. В пачке с одним корнем в цепочку попадает только корень, а запись и путь до корня держит субъект. Подпись в 15 712 байт делится на сотни записей, а каждая запись в сегменте добавляет к цене пачки свои байты. Цена аттестации равна тарифу записи, то есть базовой плате за байт, которая сжигается. Отдельной платы за свидетельство нет. От спама защищает вес выдавшего: свидетельства выдавших без веса почти ничего не дают.

**Проверка без чужой базы.** Субъект носит доказательства с собой: свидетельство, путь Меркла, транзакцию пачки и заголовок блока. Путь он получает вместе со свидетельством или вычисляет по листьям сегмента и хранит сам, потому что узлы вправе выбросить сегмент. Проверяющий выполняет четыре шага. Подпись: пачку подписали ключи, текущие у выдавшего на высоте записи. Включение: путь от свидетельства сходится к корню пачки, а пачка лежит в блоке под заголовком. Статус: ячейка выдавшего на этой высоте жива, что видно по корню состояния в заголовке. Вес: он считается по опубликованной формуле. Первые три шага не требуют доверия ни индексатору, ни выдавшему, поэтому репутацию субъекта можно проверить, даже если индексаторы исчезли.

**Открытые записи и обязательства.** Аттестации по умолчанию открыты. В цепочку попадают только хэши и метаданные, а код, промпты, данные проекта и секреты в неё не попадают. Открытая запись всё же раскрывает, какой агент с каким проектом работал, поэтому приватный проект публикует только обязательство, то есть хэш аттестации. Полное свидетельство получает субъект и раскрывает его тому, кому хочет, а проверяющий сверяет его с обязательством. Публичный индексатор учитывает только открытые записи. Обязательство скрывает содержимое, но не факт записи и не выдавшего. Канонический состав аттестации во многом предсказуем, и хэш одного содержимого подбирался бы перебором правдоподобных записей. Поэтому обязательство равно $H(s \,\|\, m)$, где $m$ обозначает аттестацию, а $s$ случайную 256-битную соль. Соль субъект получает вместе со свидетельством и раскрывает вместе с ним.

**Репутация.** Репутацией называется функция над аттестациями, которую детерминированно вычисляют индексаторы. Индексатор считает её как неподвижную точку графа подписанных утверждений, по схеме EigenTrust [37]. Каждая личность в этом графе и выдаёт, и получает, потому что агенты выдавшего сами работают в чужих проектах. Вес втекает в граф от затравки, а затравку задают классы доказательства личности. Кластер личностей, которые аттестуют друг друга, без притока извне и без доли в затравке получает нулевой вес. С притоком его вес ограничен кратным притока, и множитель задаёт затухание. Если на каждом шаге доля $1-a$ веса идёт по рёбрам графа, а доля $a$ возвращается к затравке, кластер без исходящих рёбер, в который снаружи ведут рёбра с весом $x$, накапливает вес $x(1-a)/a$, около $5{,}7x$ при $a = 0{,}15$. Вес выдавшего зависит от класса доказательства его личности, его возраста и того, подтверждали ли его вердикты независимые выдавшие; вердикты, которые независимые выдавшие опровергли, вес снижают. Оси продьюсера, ревьювера и верификатора считаются раздельно, а старые свидетельства весят меньше новых. Исход *не установлено* нейтрален и репутации не прибавляет и не отнимает.

Формула и её константы публикуются вместе с индексатором, и конкретные значения затухания и начальных весов классов задаёт эта публикация. Проверяющий может выбрать собственную затравку и вычислить репутацию от своих доверенных точек; скрытого списка доверенных выдавших нет. Детерминированность означает, что при одинаковых записях и одинаковой формуле получается одинаковый результат, и любой может пересчитать индексатор. Правильности самой формулы она не доказывает.

**Споры.** Спором называется подписанное возражение без арбитража. Субъект публикует возражение со ссылкой на аттестацию, выдавший может опубликовать поправку или отзыв своей записи, а индексаторы показывают свидетельство и возражения рядом. Оспоренное *отклонено* весит меньше, и полный вес ему возвращает только повторное подтверждение выдавшего. Арбитра в цепочке нет: каждая сторона отвечает своей подписью и своей историей.

**Деньги и репутация.** Дизайн не связывает деньги и репутацию. Аттестация стоит только тариф записи, оплату работы задаёт политика проекта, а цепочка даёт валюту расчётов. Репутацию нельзя купить, заложить или потерять как залог, и монета не начисляется за число аттестаций. Связь между ними допустима только там, где понятны её цена и последствия, и это открытый вопрос дизайна.

**Допуск остаётся у проекта.** Переносится история подписанных утверждений, а право работать выдаёт принимающий проект по своей политике. Например, работе нового агента проект назначает двухтрёх независимых ревьюверов, а работе агента с историей одного; эти числа иллюстрируют политику и порогов не задают. Репутация не создаёт отсутствующей независимости: личности одного владельца остаются одним административным доменом, а общее семейство модели остаётся источником общих слепых зон.

## 8. Ценность: деньги, плата и эмиссия

**Деньги.** Монеты лежат в UTXO, адрес равен хэшу условия траты, и адреса одноразовые. Swarma Chain служит слоем расчётов и личностей: подписанных объектов в блоке мало (раздел 14), поэтому расчёты медленные, и для мелких покупок цепочка не предназначена. Чаевые автору служат нативной записью. Клиент или спонсор может оплачивать записи пользователей, и спонсор виден в записи.

**Размер блока и плата.** Правило размера блока целиком зашито в генезис (табл. 3), без голосований и без хардфорков ради размера. Правило учитывает два урока Monero: короткая медиана позволяла экспоненциальный рост размера, а место до зоны без штрафа было почти бесплатным, и его дёшево заливали. Базовая плата за байт меняется за блок пропорционально отклонению от цели, не больше чем на $12{,}5\,\%$, как в EIP-1559 [13], и сжигается. Минимальная плата привязана к награде, поэтому заливка стоит денег с первого байта. Рост цели требует устойчиво высокой платы, так что короткий всплеск спроса цель не двигает. У сегмента данных свой предел, поэтому платежи, имена и события личностей не вытесняются потоком записей. Срока обслуживания правило не обещает: каждая транзакция ждёт места в блоке.

**Таблица 3.** Правило размера блока, зашитое в генезис. Числа служат кандидатами для моделирования.

| Часть | Значение |
|---|---|
| Старт | Цель $T_0 = 512$ КБ на 60-секундный блок; блок не больше $2T$ |
| Плата | Базовая плата за байт меняется за блок пропорционально отклонению от цели, не больше чем на $12{,}5\,\%$, и сжигается; полный по цели блок стоит не меньше $R/16$, где $R$ награда за блок |
| Рост | Цель растёт, только когда базовая плата выше двойного минимума, и не быстрее $\times 2$ в год |
| Спад | Цель падает, только когда блоки полупустые при минимальной плате, и не быстрее $\times \frac{1}{2}$ в год; ниже $T_0$ не опускается |
| Потолок | Цель не выше $4$ МБ $\times\, 2^{y/4}$, где $y$ возраст сети в годах |
| Состав | Вес включает все байты, подписи без скидки; социальный сегмент не больше половины веса; пересчёт на каждом блоке в целых числах |

**Эмиссия.** Комиссии почти бесплатного текста дают слабую основу бюджета безопасности. Если поднять их, публикации подорожают, а если держать низкими без другого дохода, вычислительная защита недополучит оплату. Поэтому выбрана хвостовая эмиссия по образцу Monero [59]: награда плавно уменьшается вдвое каждые три года от 64 условных монет за блок и с пятнадцатого года остаётся постоянной, 2 монеты за блок. Числа графика служат кандидатами, а расчёт приведён в разделе 14. Эмиссия оплачивает продолжение работы сети и доходности держателю не обещает. Запуск честный: без премайна и без эмитента, доля фонда разработки равна нулю. Майнер получает награду, приоритетные чаевые и половину платы за имена; базовая плата за байт сжигается.

**Сервисные узлы.** Сервисным узлом называется оператор полезной сетевой функции: индексирования, раздачи медиа, разрешения имён или хранения старых сегментов. Власти над блоками у него нет, и консенсус не вводит ни мастернод, ни залоговых барьеров, ни кворумов. Оплату сервисных узлов дизайн связывает с проверяемой работой. Случайный пир запрашивает кусок сегмента, разрешение имени или подписанный хэш состояния индекса и проверяет ответ, а узел, не прошедший проверку, оплаты не получает. Параметры этого механизма составляют открытое пространство дизайна: доля награды, наличие залога, частота проверок и ответственность за систематическую ложь.

## 9. Консенсус и майнинг

**Собственный RandomX.** Майнеры работают на собственной конфигурации RandomX [58], которая нагружает CPU, со своей солью, своей деривацией ключа по высоте блока и своими размерами программ. Стойкость такой функции к коротким путям вычисления мы принимаем допущением Д7: это эвристика оптимизационной трудности, и угрозу ей создают специализированное железо и новые оптимизации. Своя конфигурация затрудняет немедленный перенос чужих настроек майнинга, но ботнетов и арендованных CPU не устраняет. Поэтому безопасность не оценивают по одному отсутствию известных ASIC: доли пулов наблюдаются, а параметры в первые два-три года сети меняются на плановых хардфорках.

**Без объединённого майнинга.** Объединённый майнинг [35] исключён по трём причинам. Крупный пул сети-донора атаковал бы молодую цепочку бесплатно, потому что он и так майнит. Monero сам держится на кривых, поэтому при их поломке его ценность и хэшрейт упадут ровно тогда, когда хэш-профиль Swarma Chain должен окупиться, и заёмная безопасность исчезнет в самый нужный момент. Без второй дорожки консенсус проще, а раздача монет целиком своя.

**Сложность.** Небольшая PoW-сеть чувствительна к переключающимся майнерам: они приходят на низкую сложность и уходят на высокую, а длинное окно пересчёта растягивает период после ухода хэшрейта. Сложность пересчитывает ASERT [23] в целочисленной форме с якорем в генезисе, без скользящего окна. В идеализированной записи цель блока высоты $h$ с меткой времени $t_h$ равна

$$T_h = T_a \cdot 2^{\,(t_h - t_a - 60\,\mathrm{c}\cdot(h-h_a))/\tau},$$

где $(h_a, t_a, T_a)$ задаёт якорь, а $\tau$ называется полупериодом: каждые $\tau$ отставания от расписания цель удваивается, то есть сложность падает вдвое. Кандидат полупериода равен двум часам. Метки времени строго монотонны, допуск в будущее равен 300 секундам, минимальной сложности на основной сети нет.

**Рой и есть майнеры.** Хосты Swarma Runtime по желанию майнят простаивающим CPU: под отдельной системной учёткой, с низким приоритетом и лимитом cgroup, с сетью только для p2p цепочки. Тысяча таких хостов даёт базу хэшрейта, которая не уходит при росте сложности. Майнеры объединяются в децентрализованный пул в духе P2Pool [55] с порогом выплат, потому что мелкие выходы съела бы цена их траты. Шаром называется решение облегчённой цели, которым майнер доказывает пулу свою работу.

**PoW-билеты допуска.** Публичная точка присоединения протокола принимает заявки от незнакомых нод, поэтому заявка стоит работы [20, 2]. Нода решает заголовок блока над свежей вершиной на облегчённую цель и прикладывает его как PoW-билет допуска. В коинбейсе билета закреплены личность,

проект, роль, эпоха и дайджест политики, поэтому билет относится к одной заявке. Если заголовок попал и в цель блока, это блок. Билет служит шаром: работа допуска охраняет цепочку, а свежая вершина служит вызовом без внешнего маяка случайности, и решить билет до её появления нельзя.

## 10. Финальность: штраф поздней ветки

Молодая CPU-сеть хэшрейтом не защитится: большинство можно арендовать. Время арендовать нельзя, и на этом строится выбор ветки. Правило продолжает линию штрафа за задержанную публикацию, предложенного для Horizen [25], где задержка ветки добавляет ей штраф, отсчитанный в блоках. Кратное требование к работе ветки идёт от MESS, который Ethereum Classic ввёл после атак 2020 года [31] и позднее отключил [32].

Узел помнит монотонное время первого получения каждого блока. Пусть конкурирующая ветка расходится с цепочкой узла в точке, после которой первый блок он получил $\Delta$ назад. Узел переходит на неё, только если её работа после точки расхождения больше работы его текущей цепочки после той же точки в $P(\Delta)$ раз:

$$P(\Delta) = \begin{cases} 1, & \Delta \le 10\ \text{мин}, \\ \min\bigl(32,\ 2^{(\Delta - 10\ \text{мин})/1\ \text{ч}}\bigr), & 10\ \text{мин} < \Delta < 24\ \text{ч}, \\ \infty, & \Delta \ge 24\ \text{ч}. \end{cases}$$

Первые 10 минут штрафа нет, поэтому обычные форки он не затрагивает. Дальше штраф удваивается каждый час до потолка 32, а через 24 часа реорганизаций нет. Параметры служат кандидатами, и симуляция проверяет их вместе с ASERT и правилами времени.

Цену двойной траты мы оцениваем в простой модели. Блоки приходят пуассоновским потоком с общей интенсивностью один блок в минуту, сложность постоянна, и влияние ASERT не учитывается. Атакующий с долей хэшрейта $\alpha$ с момента первого блока с платежом майнит тайную ветку от точки перед этим блоком, а получатель выдаёт товар через время $w$ после этого блока. Атакующий показывает ветку в момент $w$. К этому моменту у честной ветки после точки расхождения $H \sim \text{Pois}((1-\alpha)w)$ блоков, у тайной $A \sim \text{Pois}(\alpha w)$ блоков, и узлы переходят на тайную ветку, только если $A > P(w)\,H$. По средним числам блоков это даёт порог $\rho > P(w)$, где $\rho = \alpha/(1-\alpha)$, то есть $\alpha > P(w)/(1+P(w))$. На пороге тайная ветка побеждает примерно с вероятностью одна вторая, а ниже порога вероятность падает, но остаётся заметной. Таблица 6 даёт долю хэшрейта, при которой вероятность успеха достигает 50, 10 и 1 %. В расчёте показ ветки в заранее выбранный более поздний момент вероятности успеха не повышал: до потолка штраф растёт вдвое за час, и отношение работы двух веток его не догоняет. Показ при первом выполнении условия после $w$ повышает вероятность успеха на доли процентного пункта. Без штрафа атакующий с долей больше половины рано или поздно побеждает при любом ожидании [45, 54].

**Слабая субъективность.** Плата за отказ от поздних реорганизаций называется слабой субъективностью [12]. Новый узел стартует с подписанного checkpoint и снимка состояния UTXO, имён и личностей; сырые социальные сегменты для этого не нужны. Весь снимок, включая набор UTXO, узел сверяет с корнем состояния в заголовке (раздел 3). Checkpoint подписывают хэш-ключи сопровождающих, и он публикуется в релизе и на зеркалах. Узел, вернувшийся после долгого отсутствия, сверяет вершины последних суток у нескольких пиров.

**Разделение сети.** Штраф мешает сойтись и разделённой сети. Если две части с близким хэшрейтом держат разные ветки дольше 10 минут, ни одна не набирает перевеса в $P(\Delta)$ раз. Поэтому узел, получивший позднюю ветку, работа которой после точки расхождения сравнима с работой его цепочки, поднимает тревогу и не считает окончательными записи после точки расхождения. Разделение сводит следующий подписанный checkpoint: узлы части, ветка которой в него не вошла, переходят на ветку checkpoint. Если разделение длилось дольше суток, этот переход идёт через отсечку, где $P = \infty$, то есть только по подписи сопровождающих, поэтому при сведении разделения подписанты checkpoint служат опорой доверия и для работающих узлов (Д5). Задержку окончательности ограничивает интервал между подписанными checkpoint: если checkpoint выходят только с релизами, записи после точки расхождения ждут следующего релиза.

**Без второго консенсуса.** Финальность на кворумах по образцу ChainLocks [7] исключена: второй консенсус на пороговых BLS противоречил бы хэш-профилю, а его задачу решает штраф поздней ветки. Три усиления ядро не включает, и дизайн держит их в резерве. Последовательная композиция двух CPU-функций с разными узкими местами ограничила бы выигрыш от прорыва в одной из них примерно двукратным, если обе функции стоят примерно одинаково. Нотариальная отметка вершины в Bitcoin помогла бы новым узлам и разбору разделений, а выбор ветки от неё бы не зависел. STARK-агрегация подписей [4], тоже на одних хэшах, сняла бы цену их размера.

**Таблица 4.** Что слои Swarma берут из цепочки и где проходит граница.

| Слой | Что берёт из цепочки | Граница |
|---|---|---|
| Swarma Runtime | Ячейки агентов с ключами у ключника; журнал ключевых событий в реестре владельца с тем же идентификатором; ссылки памяти на ячейки | Происхождение ключа не доказывает честности агента |
| Swarma Protocol | Личности нод и подписантов манифеста; пачки аттестаций исхода; хэш блока для выбора ревьювера; PoW-билеты допуска; закрепление голов журнала | Контроль над внешней сущностью не доказывает независимости ошибок |
| Swarma Chain | Ячейки, имена и корень их состояния в каждом заголовке | Общие записи не заменяют политику допуска принимающего проекта |

## 11. Лёгкие клиенты и индексаторы

**Заголовки от checkpoint.** Заголовок с 64-байтными обязательствами, включая корень MMR, весит около 414 байт (раздел 14), а проверка RandomX на телефоне тяжела. Лёгкий клиент начинает проверку заголовков с последнего подписанного checkpoint и всю историю не хранит. После checkpoint он выбирает ветку наибольшей работы и проверяет её доказательство работы выборкой заголовков по схеме FlyClient [10], без проверки RandomX у каждого заголовка. Выборка опирается на корень MMR в заголовке: каждый выбранный заголовок клиент проверяет путём до корня MMR в заголовке вершины ветки, поэтому выбранные заголовки принадлежат той ветке, работу которой клиент проверяет. ASERT меняет цель с каждым блоком, поэтому каждый узел MMR фиксирует ещё суммарную работу своего поддерева, и FlyClient выбирает заголовки по доле накопленной работы, как требует его вариант для переменной сложности. В заголовке при этом остаётся один 64-байтный корень MMR, и размер заголовка не меняется. Штраф поздней ветки лёгкому клиенту недоступен: штраф требует времени первого получения каждого блока, а клиент часто бывает офлайн. Одной наибольшей работы поэтому мало: арендатор большинства может показать клиенту более тяжёлую позднюю ветку, которую полные узлы отвергли. Клиент дополнительно спрашивает $k$ независимых полных узлов о предке своей вершины на глубине $d$ и принимает ветку, только если все они называют тот же блок. Полные узлы применяют штраф и отвергнутую позднюю ветку не подтверждают. Если ответы расходятся, проверка на клиенте даёт исход *не установлено*. Гарантия лёгкого клиента слабее гарантии полного узла: она держится на выборке, которая находит поддельную работу с высокой вероятностью, и на том, что опрошенные полные узлы не сговорились с противником и не изолируют клиента. На iOS клиент работает в тонком режиме, без архивирования.

**Живая ячейка по корню.** Корень состояния в каждом заголовке позволяет лёгкому клиенту проверить по доказательству против этого корня, что ячейка выдавшего была жива на высоте записи. Вместе с путём Меркла пачки и подписью это закрывает три первых шага проверки аттестации (раздел 7) на устройстве с одними заголовками, в пределах гарантии лёгкого клиента, разобранной выше.

**Фильтры.** Лёгкий клиент читает компактные фильтры блоков по образцу BIP 157/158 [53, 52]. В BIP 158 консенсус фильтр не фиксирует, а Swarma Chain кладёт обязательство фильтра в заголовок. Фильтр для социальных записей строится по четырём ключам: тип записи, автор, ссылка на запись и тема. Так клиент ищет записи выбранного автора или ответы на известную запись, не отдавая серверу список подписок. Фильтр указывает на возможное совпадение, а полученные данные проверяются по заголовку. Запись, которую больше никто не раздаёт, фильтр не вернёт: потеря сегментов остаётся потерей истории.

**Проверяемый индекс.** Индексатор отдаёт производное состояние через GraphQL с подписками: ленты по личности и по имени, граф подписок, деревья ответов, разрешение указателей, репутацию по опубликованной формуле и доказательства включения. На заданной высоте он публикует хэш своего состояния. Клиент сравнивает ответы нескольких индексаторов на одной высоте при одинаковых правилах вывода. Совпадение подтверждает согласованность канонического вывода, а правильности произвольной рекомендации не доказывает. Точную схему задаёт спецификация индексатора.

## 12. Как рантайм и протокол используют цепочку

Рантайм строится в расчёте на Swarma Chain, и через цепочку связаны личность агента, его свидетельства и переносимая память (табл. 4).

**Рантайм.** Ключник задаёт личность агента стабильным путём вида назначение, продукт, проект, агент; путь отличается от домашнего каталога и от UID пользователя ОС. Учётку можно пересоздать, а агента перенести на другую машину, и личность при этом не меняется. Ключи ячейки ключник выводит хэш-функцией от пути и номера ротации и подписывает по запросу агента. Инъекция в промпт действует возможностями процесса агента, поэтому секрет не лежит в доступном ему файле. Честности запросов подписи ключник при этом не обеспечивает: сообщение, подписанное захваченным агентом, остаётся

неправильным сообщением. Реестр владельца ведёт журнал ключевых событий до публикации в цепочку (раздел 5). Память агентов называет издателя, субъекта и отправителя записей идентификаторами ячеек, а представительство опирается на грант в реестре владельца; недоступный реестр никогда не считается выданным грантом. Поэтому переезд агента на другую машину или в другой рантайм не рвёт ни истории, ни памяти: меняются ключи и место исполнения, а ячейка остаётся.

**Протокол.** Личность ноды задаёт её ячейка, а короткоживущие ключи ноды удостоверяет эта личность. Ревизии подписываются ключом ноды в формате SSH-подписи git, и ключ допустим, только если его удостоверила подтверждённая личность и нода назначена на задачу. Аттестации исхода из журнала проекта выпускаются пачками в формате цепочки, и их корни публикуются в Swarma Chain. Манифест федерации подписывает личность цепочки, а доказательства идентичности организации привязываются к этой личности аттестациями. Головы журналов протокола закрепляются в цепочке, и записи на Ed25519 сохраняют доказательную силу на дату наблюдения.

Выбор ревьювера опирается на хэши цепочки. Кор-нодой называется узел, который держит журнал проекта на кворуме. Протокол замораживает множество допустимых ревьюверов до появления хэша кандидата. Случайность равна хэшу раскрытий луковиц хэшей кор-нод вместе с хэшем блока цепочки, вышедшего после фиксации кандидата. Ревьювер вычисляется из идентификатора задачи, хэша кандидата и случайности, и валидатор пересчитывает результат. Публичная точка присоединения принимает заявку с PoW-билетом (раздел 9) и ключа подписи допуска не держит: точка ставит заявку в очередь, а допуск подписывает проект.

## 13. Анализ безопасности

Для каждой цели раздела 2 мы называем аргумент, допущения и предел.

**Ц1: подделка состояния.** Каждое изменение денег, имён и ячеек требует пары подписей SLH-DSA, а ключи для проверки берутся из самого состояния: из условия траты выхода или из поля ячейки. Противнику П1, сломавшему кривые, пейринги или решётки, подделывать нечего, потому что ни одна проверка консенсуса на них не опирается. Противнику, сломавшему одно семейство хэшей, нужна вторая половина пары, которая держится на другом семействе, а обязательства тем же способом требуют коллизии в обоих семействах. Противник П2 с большинством хэшрейта ничего не подделывает, он только упорядочивает. Допущение: Д1. Предел: поломка обоих семейств хэшей ломает профиль целиком.

**Ц2: перехват личности.** Вор текущих ключей ячейки агента (П3) подписывает записи, но не может ни провернуть ячейку, ни отозвать её: ротация требует следующих ключей, спрятанных за хэшем, а трата родителем и терминальный отзыв требуют ключей ячейки владельца. Ключами самой ячейки он самое большее приостанавливает её, а приостановку снимает родитель. Старое снятие приостановки повторить нельзя: оно названо на значение счётчика трат родителем, которое уже пройдено, поэтому приостановку, поставленную рантаймом, вор повтором не отменит. Вор мастер-секрета знает и следующие ключи агентов, поэтому он может провернуть ячейку и записать обязательство на ключи, которые знает только он. Владелец отвечает тратой родителем, которая не привязана к версии ячейки: ротации вора, включённые раньше, её не отменяют, а в одном блоке она побеждает ротацию. После неё ротации ключами, объявленными до неё, недействительны, и у вора нет ни текущих, ни следующих ключей новой версии. Восстановление поэтому завершается первой включённой тратой родителем, и гонка комиссий его не откладывает; задержать включение может только устойчивое большинство, которое задерживает любые записи (остаточный риск). Повтор этой траты ячейку не откатывает: трата действительна только для значения счётчика $n{+}1$, после её включения счётчик равен $n{+}1$, а ротации вора его не меняют. Так же недействителен повтор траты родителем из прошлого восстановления, и ячейку к ключам $K^*$ того восстановления не вернуть, даже если они с тех пор утекли. Отзыв не даёт вору второго пути: терминальный отзыв подписывает только родитель, и личности агентов вор не уничтожит. Записи, которые вор подписал до высоты траты родителем, проходят проверку подписи, и владелец оспаривает их возражениями. Допущение: Д6. Предел: кража ключей ячейки владельца выходит за Д6. Эти ключи тратят ячейку каждого агента и подписывают её терминальный отзыв, поэтому вор перехватывает личности агентов, а отозванные им ячейки не вернуть. Ячейку самого владельца вор не отзывает, потому что её отзыв подписывает только ключ восстановления. Владелец возвращает свою ячейку тратой ключом восстановления и затем тратит ячейки агентов, после чего ключи, которые держал вор, больше ничего не тратят. Потеря и ключа владельца, и ключа восстановления делает личность невосстановимой.

**Ц3: одна версия.** Двуличный владелец (П4) в реестре одного владельца может показать разные истории разным проверяющим. В цепочке ячейка лежит в общем состоянии, корень которого входит в каждый заголовок, поэтому два честных узла на одной ветке видят одну версию. Разные ветки возможны до созревания: противник П2 может реорганизовать свежую историю работой в пределах штрафа,

но не после 24 часов; после 24 часов ветку узла меняет только подписанный checkpoint, сводящий разделение. Допущения: Д3, Д5. Предел: узел под eclipse-атакой (П7) видит ветку противника до получения честной, а узел, вернувшийся после суток отсутствия, полагается на сверку вершин у нескольких пиров. При разделении под штрафом (предел Ц5) узлы поднимают тревогу, и одну версию восстанавливает следующий checkpoint. Эта версия держится на подписантах checkpoint (Д5): они выбирают, какая часть разделённой сети сохраняет свою историю, и противник с ключами сопровождающих вместе с поздней веткой сравнимой работы решает, какая версия ячейки останется. Лёгкий клиент получает Ц3 в пределах своей гарантии (раздел 11).

**Ц4: проверка аттестации.** Подпись проверяется по ключам выдавшего на высоте записи, включение по пути Меркла до корня пачки и заголовка, статус по корню состояния в заголовке. Ни индексатор (П6), ни выдавший в этих трёх шагах не участвуют. Индексатор может скрыть запись или подставить другую формулу. Скрытие ограничено тем, что субъект носит доказательства сам, а подмену формулы обнаруживает пересчёт, потому что вывод детерминирован. Против фабрики личностей (П5) действует вес. Кластер, который аттестует сам себя, без внешнего притока и без доли в затравке веса не получает, с притоком получает его ограниченное кратное (раздел 7), а каждая личность и каждая запись стоят платы. Предел: доказательство включения не доказывает полноты выдачи, и отсутствие записи в ответе не значит, что её не было. Корректная подпись не делает истинным утверждение внутри свидетельства. Симметричная функция репутации не бывает устойчивой к множественным личностям, а асимметричная функция потока от затравки устойчива к ним только при условиях, которые формулируют Ченг и Фридман [14]. Формула Swarma асимметрична, и выполняет ли она эти условия, зависит от её опубликованных констант, поэтому репутация снижает риск сговора и создания личностей, но не исключает его. На лёгком клиенте статус выдавшего проверяется по корню ветки, которую подтвердили $k$ полных узлов (раздел 11).

**Ц5: двойная трата.** При доле противника $\alpha$ и ожидании получателя $w$ тайная ветка побеждает с вероятностью около половины при $\alpha = P(w)/(1 + P(w))$, а вероятность 1 % достигается при меньшей доле (табл. 6). Штраф считается по монотонным часам узла и от меток блоков не зависит, поэтому сдвиг меток его не обходит. Метки влияют только на ASERT, где их ограничивают строгая монотонность и допуск 300 секунд в будущее. Штраф не меняет порогов эгоистичного майнинга [22]: стандартная стратегия обычно публикует придержанные блоки в ответ на ближайший честный блок, в пределах первых 10 минут, где $P = 1$, а придерживание дольше 10 минут требует кратного перевеса. Допущения: Д2, Д3, Д4, Д7. Предел: в первые 10 минут $P = 1$, и действует обычная гонка ветвей [54], поэтому получатель крупного платежа ждёт часы. Второй предел касается разделения сети. Если две части сети с близким хэшрейтом держат разные ветки дольше 10 минут, каждой части для перехода нужен перевес работы в $P(\Delta)$ раз, который растёт вдвое за час, и честные части такого перевеса не набирают. Сетевой противник П7 с небольшим хэшрейтом может вызвать такое разделение, если покажет конкурирующую ветку части узлов у границы 10 минут. Узлы распознают разделение по поздней ветке сравнимой работы, поднимают тревогу и не считают окончательными записи после точки расхождения, а одну ветку восстанавливает следующий подписанный checkpoint (раздел 10). Получатель видит тревогу и ждёт checkpoint, поэтому платой за разделение служит задержка окончательности; ту же задержку может вызвать и противник, показавший позднюю ветку сравнимой работы. Эту задержку ограничивает только интервал между подписанными checkpoint (раздел 10), и его значение работа не задаёт. Сведение разделения добавляет допущение Д5 к допущениям Ц5: подписанты checkpoint выбирают, какая часть сети побеждает, поэтому скомпрометированный ключ сопровождающих вместе с поздней веткой сравнимой работы решает исход разделения и с ним исход двойной траты в нём. Симуляция штрафа должна измерить, при каких задержках сети, долях хэшрейта и моментах показа ветки разделение возникает и сколько оно длится.

**Ц6: цена записи.** Минимальная базовая плата привязана к награде так, что полный по цели блок стоит не меньше $R/16$. Заливка поэтому стоит денег с первого байта, а при переполнении блоков базовая плата растёт на 12,5 % за блок. Социальный сегмент занимает не больше половины веса, и платежи, имена и события личностей конкурируют за свою половину. Предел: подпись спама не останавливает, потому что спамер тоже подписывает свои записи, и долларовую цену записи протокол не обещает. Видимость и хранение принятого спама решают клиенты, кураторы и архивисты.

**Ц7: дата наблюдения.** Голова журнала протокола или реестра, закреплённая в блоке, доказывает, что журнал в этом состоянии существовал не позже этой высоты, и это доказательство держится только на хэшах. После поломки кривых противник подделает подписи живых ключей, но не перенесёт поддельную запись в прошлое закреплённой головы. Допущения: Д1, Д5. Предел: записи между поломкой и отказом политики от класса ключей по подписи не проверить, а короткоживущие листья противник подделывает на время их действия.

**Случайность выбора.** Хэш раскрытий луковиц хэшей и хэш последующего блока никто не подбирает

**Таблица 5.** Доля подписи на одну запись пачки и длина пути, который держит субъект пачки с одним корнем, байт.

| Записей в пачке $N$ | 16 | 64 | 128 | 256 | 1024 | 4096 |
|---|---|---|---|---|---|---|
| Доля парной подписи, $15\,712/N$ | 982 | 246 | 123 | 61 | 15 | 4 |
| Путь Меркла, $64\lceil\log_2 N\rceil$ | 256 | 384 | 448 | 512 | 640 | 768 |
| Доказательство и доля подписи | 1238 | 630 | 571 | 573 | 655 | 772 |

после фиксации кандидата. Предел: участник, раскрывающий последним, может не раскрыть прообраз, а майнер может отбросить найденный блок с неудобным хэшем ценой награды за него. Каждое такое действие даёт противнику ещё одну попытку, и майнер может повторять отказ всякий раз, когда находит блок. За $m$ попыток при $n$ допустимых ревьюверах противник получает нужного ему ревьювера с вероятностью около $1-(1-1/n)^m$, так что смещение ограничено и растёт с числом попыток.

**Предел дизайна.** Один предел дизайн называет явно. Разделённая под штрафом сеть сама до суток не сходится: узлы поднимают тревогу, записи после точки расхождения не окончательны, и одну ветку восстанавливает следующий подписанный checkpoint (Ц3, Ц5). Сводя разделение, узлы доверяют подписантам checkpoint (Д5).

**Гарантия лёгкого клиента.** Лёгкий клиент получает отсечку реорганизаций только через согласие $k$ независимых полных узлов, и его гарантия слабее гарантии полного узла (Ц3, Ц4).

**Остаточный риск.** Устойчивое большинство может часами задерживать записи. Отдельный узел можно изолировать eclipse-атакой. Атака на небольшую сеть дешевле, чем на Bitcoin, и параметры финальности рассчитаны именно на арендованное большинство.

## 14. Анализ параметров

Числа ниже получены расчётом из кандидатных параметров дизайна. Окончательные значения генезиса задаёт моделирование сложности, размера блока, эмиссии, штрафа поздней ветки и цены атаки.

**Подписи и пропускная способность.** Парная подпись весит 15 712 байт. Блок по цели $T_0 = 512$ КБ вмещает около $512\cdot1024/15\,712\approx33$ подписанных объектов, около одного объекта в две секунды, а блок наибольшего размера $2T_0$ вмещает около 67. При полном социальном сегменте телу остаётся половина веса, около 16 объектов на блок. Поэтому события личностей, имена и платежи редки по замыслу, а массовые записи идут пачками.

**Пачки.** В пачке из $N$ записей на одну запись приходится $15\,712/N$ байт подписи (табл. 5). В пачке с листьями в сегменте путей в цепочке нет, и запись платит за свои байты и долю подписи. В пачке с одним корнем путь в $64\lceil\log_2 N\rceil$ байт держит субъект, и он длиннее доли подписи уже примерно с $N=41$. Доказательство субъекта вместе с долей подписи наименьшее при $N=128$ и $N=256$, около 571 байта, и с дальнейшим ростом пачки растёт, потому что путь удлиняется быстрее, чем падает доля подписи.

**Цена записи.** Минимальная плата за байт равна $f_{\min}=R/(16\,T)$. Запись в 1 КБ при цели 512 КБ стоит не меньше $R/8192$, то есть около 0,0078 монеты при начальной награде в 64 условные монеты. Событие личности с парной подписью весит больше 15 КБ и стоит не меньше примерно $R/534$, около 0,12 монеты при той же награде. Это нижняя граница цены каждой новой личности, и при переполненных блоках она растёт вместе с базовой платой.

**Рост цели.** Цель растёт не быстрее чем вдвое в год, а потолок равен $4$ МБ $\cdot\,2^{y/4}$. При непрерывном наибольшем росте от 0,5 МБ цель $0{,}5\cdot2^{y}$ МБ догоняет потолок при $2^{3y/4}=8$, то есть к концу четвёртого года, на 8 МБ. Дальше её ограничивает потолок, который растёт вдвое за четыре года, примерно на 19 % в год, а наибольший блок в этот момент равен 16 МБ.

**Цепочка заголовков.** Без корня MMR заголовок с 64-байтными обязательствами весит около 350 байт, а корень MMR добавляет одно 64-байтное обязательство, так что заголовок весит около $350+64=414$ байт. При 60-секундном блоке за год выходит $525\,600$ блоков, и заголовки занимают $525\,600\cdot414$ байт, около 218 МБ в год. Лёгкий клиент хранит только заголовки от последнего checkpoint.

**Двойная трата.** Таблица 6 даёт штраф и долю хэшрейта для успеха тайной ветки в модели раздела 10. Расчёт воспроизводим точной суммой по распределению Пуассона.

**Сложность.** Поведение ASERT при $\tau=2$ ч следует из идеализированной формулы в непрерывном приближении: хэшрейт после скачка постоянен, а случайность блоков усреднена. Пусть $s(t)$ равно среднему интервалу блока, делённому на 60 секунд. Формула даёт $ds/dt=-(\ln2/\tau)(s-1)$, поэтому после ступенчатого изменения хэшрейта разрыв $s-1$ закрывается с полупериодом $\tau$ в обе стороны. После десятикратного роста хэшрейта интервал начинается с $0{,}1$ от цели, после ухода майнера с 90 % хэшрейта с десяти целей, и в обоих случаях 90 % разрыва закрываются за $\tau\log_2 10\approx6{,}6$ ч прошедшего времени.

**Таблица 6.** Штраф поздней ветки и доля хэшрейта атакующего, при которой тайная ветка побеждает с заданной вероятностью. Модель: пуассоновские блоки, один блок в минуту, постоянная сложность, показ ветки в момент $w$, победа при $A > P(w)\,H$.

| Ожидание получателя $w$ | 10 мин | 1 ч | 2 ч | 3 ч | 5 ч 10 мин | 24 ч |
|---|---|---|---|---|---|---|
| $P(w)$ | 1 | 1,78 | 3,56 | 7,13 | 32 | $\infty$ |
| Порог по средним, $P/(1+P)$ | 50,0 % | 64,1 % | 78,1 % | 87,7 % | 97,0 % | невозможно |
| Доля для успеха 50 % | 52,5 % | 64,0 % | 78,0 % | 87,6 % | 96,9 % | невозможно |
| Доля для успеха 10 % | 32,6 % | 55,8 % | 72,9 % | 84,2 % | 95,5 % | невозможно |
| Доля для успеха 1 % | 18,1 % | 48,8 % | 68,4 % | 81,1 % | 94,1 % | невозможно |

**Таблица 7.** Графики эмиссии для минутного блока, условные монеты. *Число модели графика, около 150 млн; без точного помесячного шага оно воспроизводится только в пределах 136–163 млн.

| График | Начало и спад | Хвост | До хвоста | Компромисс |
|---|---|---|---|---|
| Халвинги и хвост | 50 за блок, вдвое каждые 4 года | 3,125 с 16-го года | 197 млн | Резкое падение дохода майнеров |
| Плавный спад и хвост (выбран) | 64 за блок, плавно вдвое каждые 3 года | 2 с 15-го года | 141 млн | Нужна целочисленная экспонента |
| Быстрый спад и хвост | 200 за блок, помесячное снижение | 2 примерно с 7-го года | 150 млн* | Преимущество ранних участников |

Интеграл частоты блоков по тому же приближению даёт 1440 блоков примерно за 22, 17 и 11 ч после двух-, десяти- и стократного роста хэшрейта (раздел 6). При росте примерно до 800 раз созревание имени длиннее 5 ч 10 мин, за которые штраф поздней ветки доходит до потолка 32; при большем росте имя созревает раньше потолка. Стационарный шум сложности даёт приближение Орнштейна-Уленбека для отставания от расписания: относительное отклонение сложности около $\sqrt{\ln 2/(2\tau/60\,\text{с})} \approx 5{,}4\,\%$. Симуляция проверяет эти оценки на дискретных блоках, а также скачки хэшрейта, уход крупного майнера и взаимодействие пересчёта с метками времени.

**Эмиссия.** Таблица 7 сравнивает три графика для минутного блока; монеты условные. Для выбранного плавного спада награда равна $R(y) = 64 \cdot 2^{-y/3}$ до пятнадцатого года, а выпуск до хвоста равен

$$\int_0^{15} 64\,B\,2^{-y/3}\,dy = 64\,B\,\frac{3}{\ln 2}\left(1 - 2^{-5}\right) \approx 141\ \text{млн}, \qquad B = 525\,600\ \text{блоков в год}.$$

Чуть больше половины этого выпуска приходится на первые три года ($51{,}6\,\%$), больше 90 % на первые десять. Хвост в 2 монеты за блок даёт $1\,051\,200$ монет в год, около 0,75 % накопленного к его началу предложения, и дальше эта доля уменьшается. График с халвингами выпускает до хвоста $50 \cdot 4B \cdot (1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8}) \approx 197$ млн монет. Сжигаемая базовая плата и минимальная плата, привязанная к награде, связывают график эмиссии с правилом размера блока, поэтому их числа моделируются вместе.

## 15. Смежные работы

**Цепочки с доказательством работы.** Bitcoin [45] ввёл цепочку наибольшей работы и UTXO, Ethereum [11, 61] ввёл общее состояние с контрактами, а анализ протокола Bitcoin [24] задаёт свойства общего префикса и качества цепочки при честном большинстве. Swarma Chain берёт UTXO и наибольшую работу и меняет правило выбора ветки: поздняя ветка платит за задержку кратным перевесом работы. Эгоистичный майнинг [22], eclipse-атаки [28] и анализ двойной траты [54] задают атаки, против которых рассчитаны параметры. Штраф поздней ветки продолжает аддитивный штраф в блоках из предложения Horizen [25] и кратное требование к работе из MESS [31, 32], а checkpoint следует рассуждению о слабой субъективности [12]. ChainLocks [7] дают финальность кворумом мастернод на пороговых BLS, и хэш-профиль её исключает. Объединённый майнинг разобран в [35], а RandomX [58], P2Pool [55], ASERT [23] и хвостовая эмиссия Monero [59] служат образцами компонентов.

**Хэш-подписи.** Одноразовые подписи Лэмпорта [38] и деревья Меркла [43] лежат в основе хэш-подписей. XMSS и LMS [8, 30, 41] компактнее, но держат состояние ключа, и NIST задаёт для них отдельные требования к управлению состоянием [15]; QRL [60] строит цепочку на XMSS. SPHINCS, SPHINCS$^+$ и стандарт SLH-DSA [5, 6, 50] убирают состояние ценой размера. Swarma Chain берёт SLH-DSA парой из двух семейств хэшей, чтобы поломка одного семейства не давала подделки. STARK [4] дают агрегацию на тех же предположениях. Угрозу кривым задаёт алгоритм Шора [56], а алгоритм Гровера [26] даёт перебору прообразов хэш-функции лишь квадратичное ускорение.

**Личности и прозрачность.** KERI [57] вводит самосертифицирующие идентификаторы, журнал ключевых событий и предротацию. Swarma переносит журнал в состояние консенсуса, где раздвоение исключает сама цепочка, и добавляет внешнего родителя как второй путь траты. Sidetree [9] и сеть ION [17] ближе всего к ячейке: операции личности несут обязательство на значение следующего обновления,

а их порядок задают якоря в базовой цепочке, у ION в Bitcoin; содержание операций проверяют узлы Sidetree вне консенсуса базовой цепочки. Certificate Transparency [39, 40], CONIKS [42] и Sigstore [51] делают раздвоение журналов обнаружимым для аудиторов. Иерархическая деривация [62, 29] задаёт ключи каналов рантайма.

**Имена.** Namecoin [46] поставил имена в цепочку, а его эмпирическое изучение [36] показало, что пространство имён без сдерживания скваттинга захватывается. ENS [33] задаёт имена контрактом поверх Ethereum. Handshake [27] хранит имена в выходах UTXO с ковенантами и регистрирует их аукционом со скрытыми ставками, где ставка сначала закрепляется хэшем и затем раскрывается.

**Репутация, цена работы и лёгкие клиенты.** EigenTrust [37] вычисляет глобальное доверие как неподвижную точку графа, а атака множественными личностями [18] и пределы устойчивых к ней механизмов [14] задают границы репутации. EAS [21] даёт реестр схем и аттестаций контрактами на EVM-совместимых цепочках; в Swarma Chain аттестация служит типизированной записью цепочки, а статус выдавшего проверяется по корню состояния в заголовке. Цена работы против спама восходит к [20, 2], и PoW-билеты Swarma делают эту работу шарами майнинга. Рынок платы следует EIP-1559 [13], фильтры лёгкого клиента следуют BIP 157/158 [53, 52]. FlyClient [10] сокращает проверку заголовков вероятностной выборкой по корню MMR над предыдущими заголовками; при переменной сложности узлы MMR фиксируют суммарную работу поддерева, и выборка идёт по доле работы. Swarma Chain кладёт этот корень в каждый заголовок с генезиса, а лёгкий клиент применяет выборку к заголовкам после подписанного checkpoint.

## 16. Заключение

Хэш-профиль защищает подписи и обязательства, а слабым местом Swarma Chain остаётся порядок записей в молодой сети. Штраф поздней ветки делает двойную трату после нескольких часов ожидания дорогой, но вероятность успеха ниже порога остаётся заметной, и получатель крупного платежа выбирает ожидание по таблице вероятностей. Тот же штраф мешает разделённой сети сойтись до суток, поэтому узлы поднимают тревогу, а ветку выбирает следующий checkpoint, и в этом выборе узлы доверяют его подписантам; лёгкому клиенту штраф недоступен, и он опирается на выборочную проверку работы и согласие независимых полных узлов. Восстановление личности после кражи мастер-секрета завершается первой включённой тратой родителем, потому что она не привязана к версии ячейки и побеждает ротацию в том же блоке, а терминальный отзыв подписывает только родитель; счётчик трат родителем делает повтор такой траты недействительным.

Числа генезиса, то есть цель и темпы размера блока, минимальная плата, полупериод ASERT, параметры штрафа и графика эмиссии, задаёт моделирование, а константы формулы репутации, параметры оплаты сервисных узлов и связь денег с репутацией остаются открытыми точками дизайна. Ячейка доказывает происхождение ключа, аттестация доказывает исход процедуры, а решение о допуске остаётся у проекта.

## Список литературы


[1] J.-P. Aumasson, D. J. Bernstein, W. Beullens, C. Dobraunig, M. Eichlseder, S. Fluhrer, S.-L. Gazdag, A. Hülsing, P. Kampanakis, S. Kölbl, T. Lange, M. M. Lauridsen, F. Mendel, R. Niederhagen, C. Rechberger, J. Rijneveld, P. Schwabe, and B. Westerbaan. SPHINCS$^+$: Submission to the NIST post-quantum project, v.3.1. Specification, sphincs.org, June 2022. Section 10, Tables 4, 6 and 7: single core of a 3.1 GHz Intel Xeon E3-1220 (Haswell). `https://sphincs.org/data/sphincs+-r3.1-specification.pdf`. Accessed 2026-10-25.

[2] A. Back. Hashcash – a denial of service counter-measure. `http://www.hashcash.org/papers/hashcash.pdf`, Aug. 2002. Accessed 2026-10-25.

[3] M. Bellare and D. Micciancio. A new paradigm for collision-free hashing: Incrementality at reduced cost. In *Advances in Cryptology (EUROCRYPT '97)*, volume 1233 of *Lecture Notes in Computer Science*, pages 163–192. Springer, 1997.

[4] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018. `https://eprint.iacr.org/2018/046`.

[5] D. J. Bernstein, D. Hopwood, A. Hülsing, T. Lange, R. Niederhagen, L. Papachristodoulou, M. Schneider, P. Schwabe, and Z. Wilcox-O'Hearn. SPHINCS: Practical stateless hash-based signatures. In *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 368–397. Springer, 2015.

[6] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe. The SPHINCS$^+$ signature framework. In *2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*, pages 2129–2146. ACM, 2019.

[7] A. Block. DIP 0008: ChainLocks. `https://github.com/dashpay/dips/blob/master/dip-0008.md`, Nov. 2018. Dash Improvement Proposal, status Final. Accessed 2026-10-25.

[8] J. Buchmann, E. Dahmen, and A. Hülsing. XMSS – a practical forward secure signature scheme based on minimal security assumptions. In *Post-Quantum Cryptography (PQCrypto 2011)*, volume 7071 of *Lecture Notes in Computer Science*, pages 117–129. Springer, 2011.

[9] D. Buchner, O. Steele, and T. Ronda. Sidetree v1.0.1. `https://identity.foundation/sidetree/spec/`. Decentralized Identity Foundation, ratified specification. Accessed 2026-10-25.

[10] B. Bünz, L. Kiffer, L. Luu, and M. Zamani. FlyClient: Super-light clients for cryptocurrencies. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 928–946. IEEE, 2020.

[11] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform. `https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf`, 2014. Ethereum white paper. Accessed 2026-10-25.

[12] V. Buterin. Proof of stake: How I learned to love weak subjectivity. `https://blog.ethereum.org/2014/11/25/proof-stake-learned-love-weak-subjectivity`, Nov. 2014. Ethereum Foundation Blog, 25 November 2014. Accessed 2026-10-25.

[13] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, and A. Bakhta. EIP-1559: Fee market change for ETH 1.0 chain. `https://eips.ethereum.org/EIPS/eip-1559`, Apr. 2019. Ethereum Improvement Proposal, status Final. Accessed 2026-10-25.

[14] A. Cheng and E. Friedman. Sybilproof reputation mechanisms. In *2005 ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON '05)*, pages 128–132. ACM, 2005.

[15] D. A. Cooper, D. C. Apon, Q. H. Dang, M. S. Davidson, M. J. Dworkin, and C. A. Miller. Recommendation for stateful hash-based signature schemes. NIST Special Publication 800-208, NIST, Oct. 2020.

[16] R. Dahlberg, T. Pulls, and R. Peeters. Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs. Cryptology ePrint Archive, Paper 2016/683, 2016. Nordic Conference on Secure IT Systems (NordSec) 2016.

[17] Decentralized Identity Foundation. ION: A public, permissionless, decentralized identifier network on Bitcoin. `https://github.com/decentralized-identity/ion`. Implements the Sidetree protocol as a Layer 2 overlay on Bitcoin. Accessed 2026-10-25.

[18] J. R. Douceur. The Sybil attack. In *Peer-to-Peer Systems: First International Workshop (IPTPS 2002)*, volume 2429 of *Lecture Notes in Computer Science*, pages 251–260. Springer, 2002.

[19] T. Dryja. Utreexo: A dynamic hash-based accumulator optimized for the Bitcoin UTXO set. Cryptology ePrint Archive, Paper 2019/611, 2019.

[20] C. Dwork and M. Naor. Pricing via processing or combatting junk mail. In *Advances in Cryptology – CRYPTO '92*, volume 740 of *Lecture Notes in Computer Science*, pages 139–147. Springer, 1993.

[21] Ethereum Attestation Service. EAS contracts: A free and open protocol for on-chain attestations on EVM compatible blockchains. `https://github.com/ethereum-attestation-service/eas-contracts`. SchemaRegistry and EAS contracts. Accessed 2026-10-25.

[22] I. Eyal and E. G. Sirer. Majority is not enough: Bitcoin mining is vulnerable. In *Financial Cryptography and Data Security (FC 2014)*, volume 8437 of *Lecture Notes in Computer Science*, pages 436–454. Springer, 2014.

[23] freetrader, J. Toomim, C. Culianu, and M. Lundeberg. 2020-NOV-15 ASERT difficulty adjustment algorithm (aserti3-2d). `https://upgradespecs.bitcoincashnode.org/2020-11-15-asert/`, Aug. 2020. Bitcoin Cash upgrade specification, version 0.6, 2020-08-12. Accessed 2026-10-25.

[24] J. Garay, A. Kiayias, and N. Leonardos. The Bitcoin backbone protocol: Analysis and applications. In *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 281–310. Springer, 2015.

[25] A. Garoffolo, P. Stabilini, R. Viglione, and U. Stav. Proposal to modify Satoshi consensus to enhance protection against 51% attacks: A penalty system for delayed block submission. `https://web.archive.org/web/20200217015049/https://www.horizen.global/assets/files/A-Penalty-System-for-Delayed-Block-Submission-by-Horizen.pdf`, June 2018. Horizen white paper, 14 June 2018. Archived copy; the original URL no longer resolves. Accessed 2026-10-25.

[26] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)*, pages 212–219. ACM, 1996.

[27] Handshake Project. Handshake design notes. `https://handshake.org/files/handshake.txt`. Descriptive title: the document is untitled. Covenants on UTXO outputs, blinded BID and REVEAL. Accessed 2026-10-25.

[28] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg. Eclipse attacks on Bitcoin's peer-to-peer network. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 129–144. USENIX Association, 2015.

[29] J. Hoenicke and P. Rusnak. SLIP-0010: Universal private key derivation from master private key. `https://github.com/satoshilabs/slips/blob/master/slip-0010.md`, 2016. SatoshiLabs Improvement Proposal. Accessed 2026-10-25.

[30] A. Hülsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: eXtended Merkle signature scheme. RFC 8391, IRTF, May 2018.

[31] Isaac. ECIP-1100: MESS (Modified Exponential Subjective Scoring). `https://ecips.ethereumclassic.org/ECIPs/ecip-1100`, 2020. Ethereum Classic Improvement Proposal, created 2020-09-09; status Replaced (by ECIP-1110, January 2024). Accessed 2026-10-25.

[32] Isaac. ECIP-1110: Deactivate MESS (Modified Exponential Subjective Scoring). `https://ecips.ethereumclassic.org/ECIPs/ecip-1110`, 2023. Ethereum Classic Improvement Proposal, created 2023-09-17; status Active. Accessed 2026-10-25.

[33] N. Johnson. ERC-137: Ethereum domain name service – specification. `https://eips.ethereum.org/EIPS/eip-137`, Apr. 2016. Ethereum Request for Comments, status Final. Accessed 2026-10-25.

[34] A. Joux. Multicollisions in iterated hash functions. Application to cascaded constructions. In *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 306–316. Springer, 2004.

[35] A. Judmayer, A. Zamyatin, N. Stifter, A. G. Voyiatzis, and E. Weippl. Merged mining: Curse or cure? In *Data Privacy Management, Cryptocurrencies and Blockchain Technology (ESORICS 2017 International Workshops)*, volume 10436 of *Lecture Notes in Computer Science*, pages 316–333. Springer, 2017.

[36] H. Kalodner, M. Carlsten, P. Ellenbogen, J. Bonneau, and A. Narayanan. An empirical study of Namecoin and lessons for decentralized namespace design. In *Workshop on the Economics of Information Security (WEIS 2015)*, Delft, The Netherlands, June 2015. `https://econinfosec.org/archive/weis2015/papers/WEIS_2015_kalodner.pdf`.

[37] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina. The EigenTrust algorithm for reputation management in P2P networks. In *Twelfth International Conference on World Wide Web (WWW '03)*, pages 640–651. ACM, 2003.

[38] L. Lamport. Constructing digital signatures from a one way function. Technical Report CSL-98, SRI International, Oct. 1979.

[39] B. Laurie, A. Langley, and E. Kasper. Certificate transparency. RFC 6962, IETF, June 2013.

[40] B. Laurie, E. Messeri, and R. Stradling. Certificate transparency version 2.0. RFC 9162, IETF, Dec. 2021.

[41] D. McGrew, M. Curcio, and S. Fluhrer. Leighton-Micali hash-based signatures. RFC 8554, IRTF, Apr. 2019.

[42] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 383–398. USENIX Association, 2015.

[43] R. C. Merkle. A certified digital signature. In *Advances in Cryptology – CRYPTO '89 Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238. Springer, 1990.

[44] S. Micali, M. Rabin, and S. Vadhan. Verifiable random functions. In *40th Annual Symposium on Foundations of Computer Science (FOCS '99)*, pages 120–130. IEEE Computer Society, 1999.

[45] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. `https://bitcoin.org/bitcoin.pdf`, 2008. Accessed 2026-10-25.

[46] Namecoin project. Namecoin. `https://www.namecoin.org/`. Accessed 2026-10-25.

[47] National Institute of Standards and Technology. Secure hash standard (SHS). FIPS 180-4, NIST, Aug. 2015.

[48] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS 202, NIST, Aug. 2015.

[49] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. FIPS 203, NIST, Aug. 2024.

[50] National Institute of Standards and Technology. Stateless hash-based digital signature standard. FIPS 205, NIST, Aug. 2024.

[51] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367. ACM, 2022.

[52] O. Osuntokun and A. Akselrod. BIP 158: Compact block filters for light clients. `https://github.com/bitcoin/bips/blob/master/bip-0158.mediawiki`, 2017. Bitcoin Improvement Proposal, status Deployed. Accessed 2026-10-25.

[53] O. Osuntokun, A. Akselrod, and J. Posen. BIP 157: Client side block filtering. `https://github.com/bitcoin/bips/blob/master/bip-0157.mediawiki`, 2017. Bitcoin Improvement Proposal, status Deployed. Accessed 2026-10-25.

[54] M. Rosenfeld. Analysis of hashrate-based double spending. arXiv:1402.2009, Feb. 2014.

[55] SChernykh. P2Pool: Decentralized pool for Monero mining. `https://github.com/SChernykh/p2pool`, 2021. Software repository, created 2021. Accessed 2026-10-25.

[56] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, Oct. 1997.

[57] S. M. Smith. Key event receipt infrastructure (KERI). arXiv:1907.02143, 2019.

[58] tevador. RandomX design. `https://github.com/tevador/RandomX/blob/master/doc/design.md`, 2019. Design document in the RandomX repository, first committed January 2019. Accessed 2026-10-25.

[59] The Monero Project. Tail emission. `https://www.getmonero.org/resources/moneropedia/tail-emission.html`. Moneropedia. Accessed 2026-10-25.

[60] P. Waterland. Quantum resistant ledger (QRL). `https://github.com/theQRL/Whitepaper/blob/master/QRL_whitepaper.pdf`, 2016. White paper, revised edition dated October 2016. Accessed 2026-10-25.

[61] G. Wood. Ethereum: A secure decentralised generalised transaction ledger. `https://ethereum.github.io/yellowpaper/paper.pdf`, 2025. Ethereum yellow paper, Shanghai version efc5f9a, 2025-02-04. Accessed 2026-10-25.

[62] P. Wuille. BIP 32: Hierarchical deterministic wallets. `https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki`, 2012. Bitcoin Improvement Proposal. Accessed 2026-10-25.

[63] Алексей Григорьев, Иван Опарин. Swarma. Платформа для роёв ИИ-агентов: от одной машины до открытой сети. Книга, версия 1.6, рукопись, 2026. Не опубликована.

[64] Алексей Григорьев, Иван Опарин. Swarma: проверяемые границы доверия для роёв автономных агентов. Работа, версия 6.0.1, рукопись, 2026. Не опубликована.